

Data Communications and Networking Fourth Edition

Forouzan

ANURAG UNIVERSITY

CONTENTS:

Routing, Forwarding, Routing Protocols

Praveen Kumar Reddy
Department of E.C.E

22-3 UNICAST ROUTING PROTOCOLS

A routing table can be either static or dynamic. A static table is one with manual entries.. A dynamic table is one that is updated automatically when there is a change somewhere in the Internet.. A routing protocol is a combination of rules and procedures that lets routers in the Internet inform each other of changes..

Topics discussed in this section:

Optimization
Intra- and Interdomain Routing
Distance Vector Routing and RIP
Link State Routing and OSPF
Path Vector Routing and BGP

Figure 22.13 Dynamic *routing protocols*

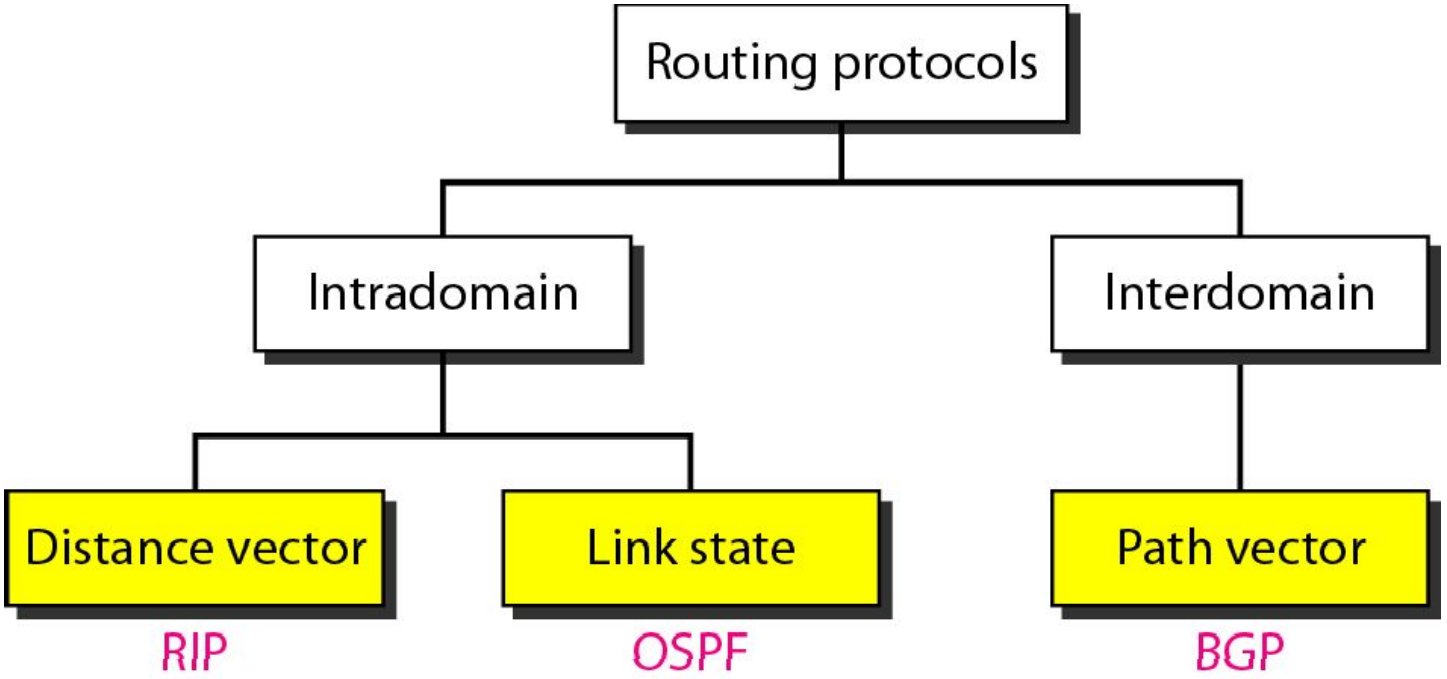
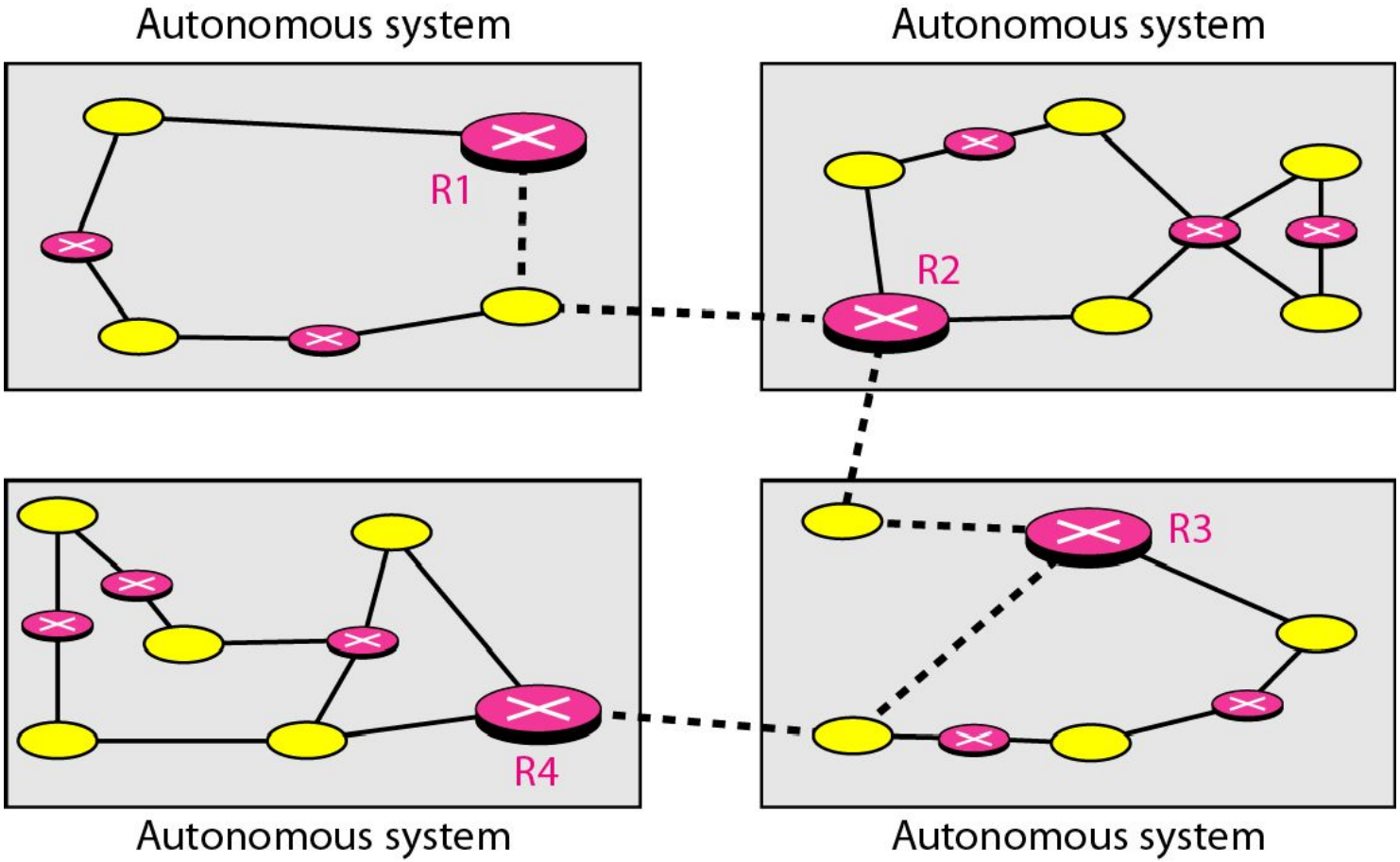


Figure 22.12 *Autonomous systems*



Intra- and Inter domain Routing

Today, an internet can be so large that one routing protocol cannot handle the task of updating the routing tables of all routers. For this reason, an internet is divided into autonomous systems. An autonomous system (AS) is a group of networks and routers under the authority of a single administration.

Routing inside an autonomous system is referred to as intra domain routing. **Routing between autonomous systems is referred to as inter domain routing**. Each autonomous system can choose one or more intra domain routing protocols to handle routing inside the autonomous system. However, only one inter domain routing protocol handles routing between autonomous systems.

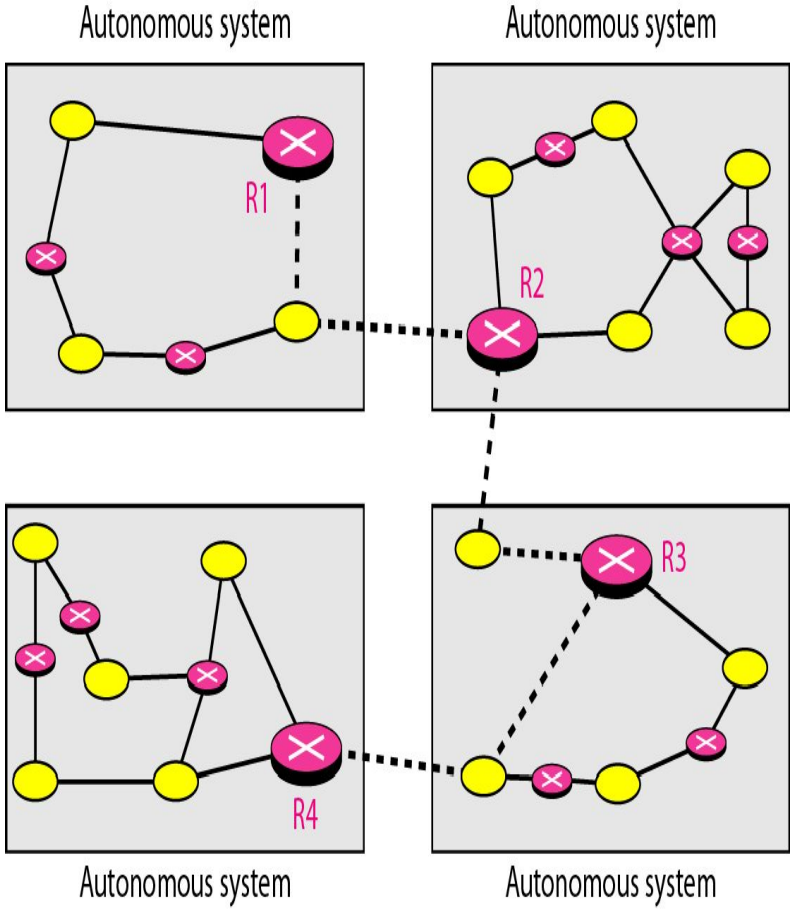
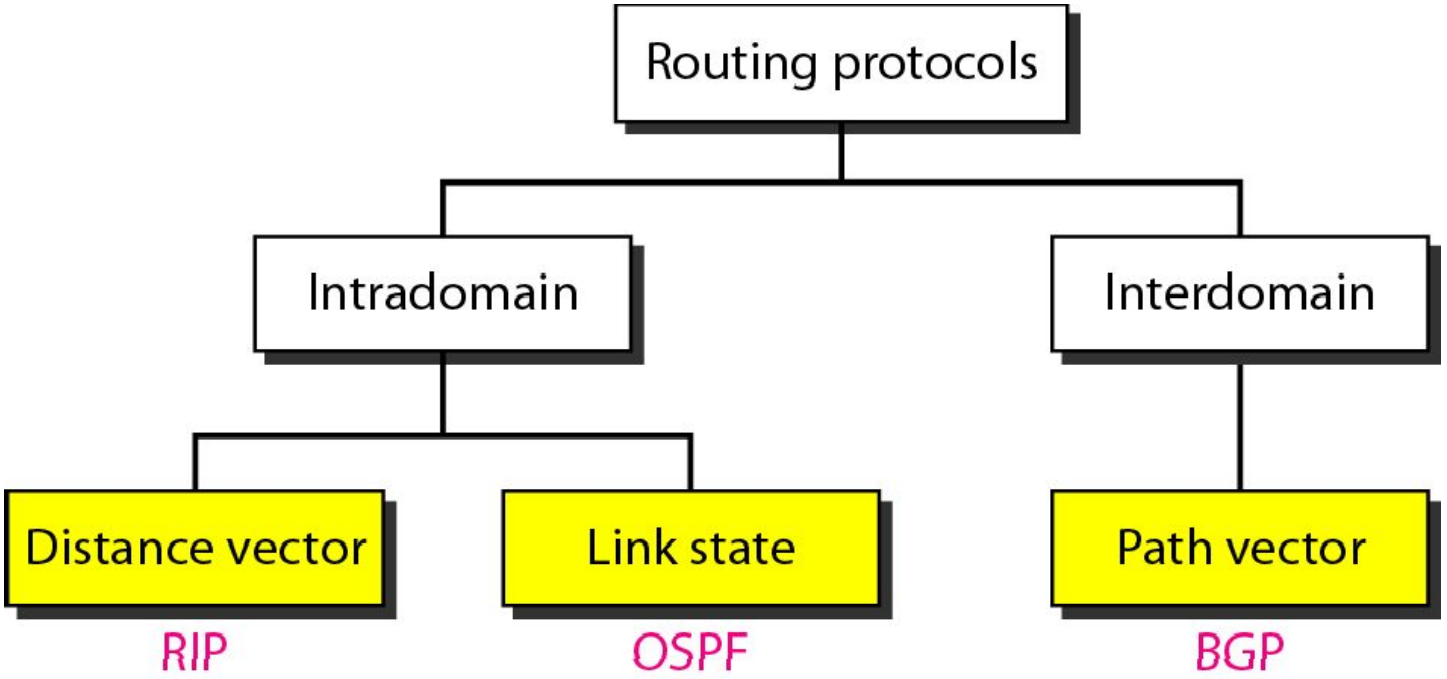


Figure 22.13 Dynamic *routing protocols*



Distance Vector Routing

In distance vector routing, the least-cost route between any two nodes is the route with minimum distance. In this protocol, as the name implies, each node maintains a vector (table) of minimum distances to every node.

The table at each node also guides the packets to the desired node by showing the next stop in the route (next-hop routing). We can think of nodes as the cities in an area and the lines as the roads connecting them. A table can show a tourist the minimum distance between cities.

.

Distance Vector Routing

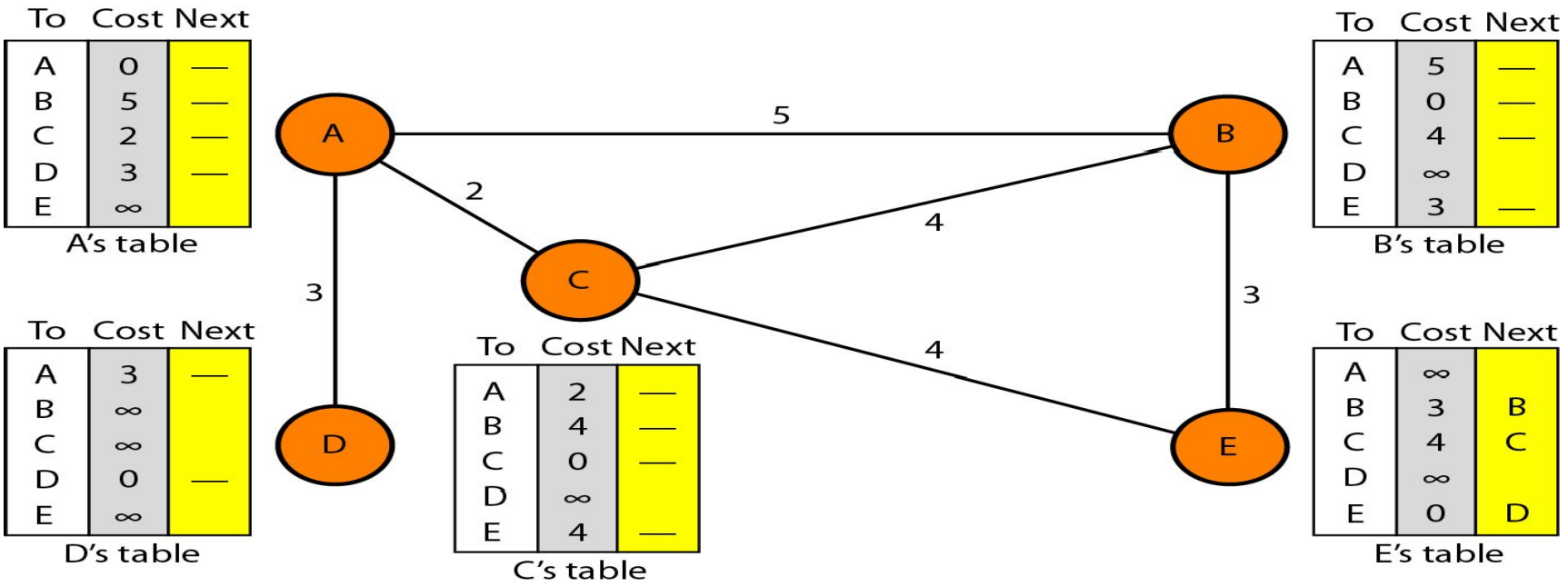
Key to understand the working of Distance Vector Routing Algorithm:

Routing only to neighbors: The router sends its knowledge about the network to only those routers which have direct links. The router sends whatever it has about the network through the ports. The information is received by the router and uses the information to update its own routing table.

Information sharing at regular intervals: Within 30 seconds, the router sends the information to the neighboring routers.

.

Step1: *Initialization of tables in distance vector routing*



each node knows how to reach any other node and the cost. At the beginning, however, this is not the case. Each node can know only the distance between itself and its immediate neighbors, those directly connected to it.

So for the moment, we assume that each node can send a message to the immediate neighbors and find the distance between itself and these neighbors. Figure shows the initial tables for each node. The distance for any entry that is not a neighbor is marked as infinite (unreachable).



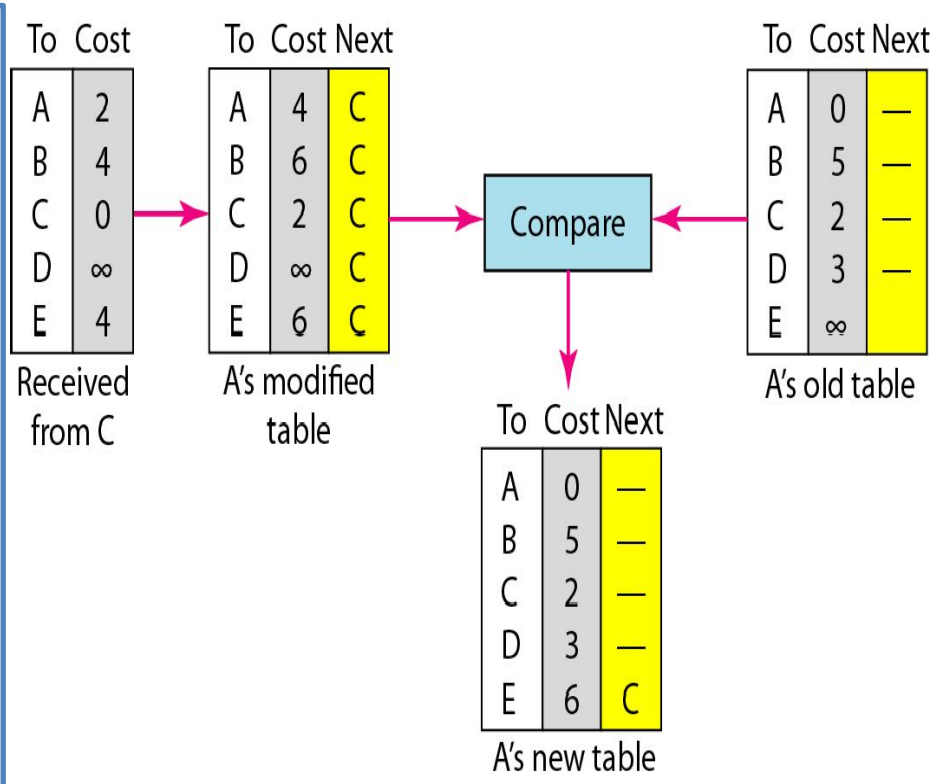
Step2:

In distance vector routing, each node shares its routing table with its immediate neighbors periodically and when there is a change.

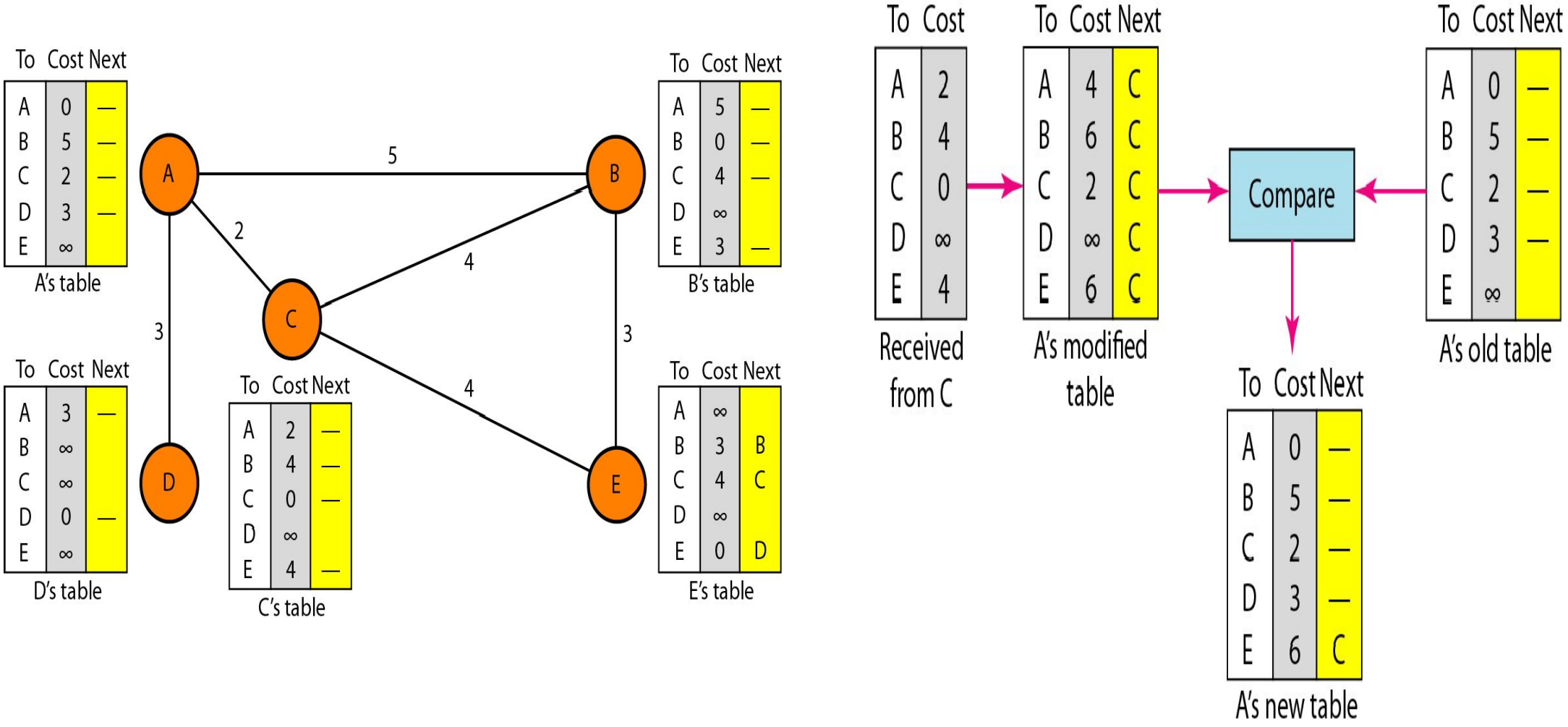
Step2: Figure 22.16 *sharing distance vector routing to neighbour*

Sharing

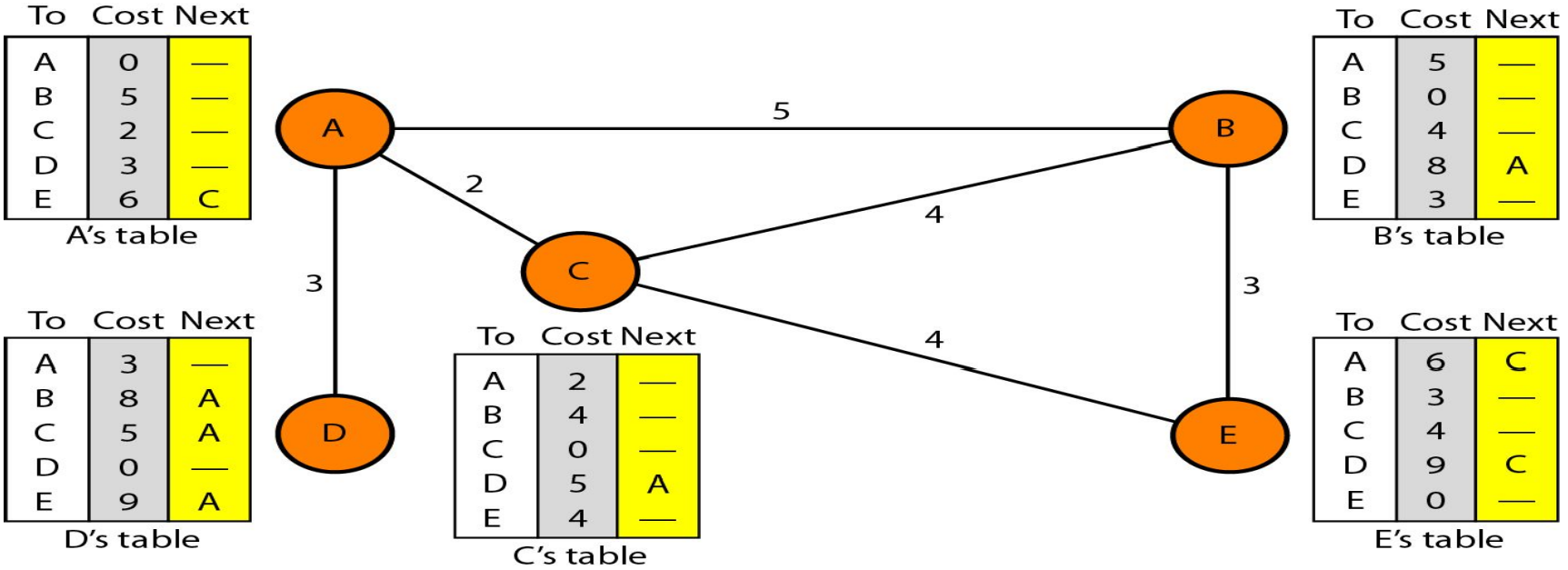
The whole idea of distance vector routing is the sharing of information between neighbors. Although node A does not know about node E, node C does. So if node C shares its routing table with A, node A can also know how to reach node E. On the other hand, node C does not know how to reach node D, but node A does. If node A shares its routing table with node C, node C also knows how to reach node D. In other words, nodes A and C, as immediate neighbors, can improve their routing tables if they help each other. There is only one problem. How much of the table must be shared with each neighbor? A node is not aware of a neighbor's table. The best solution for each node is to send its entire table to the neighbor and let the neighbor decide what part to use and what part to discard. However, the third column of a table (next stop) is not useful for the neighbor. When the neighbor receives a table, this column needs to be replaced with the sender's name. If any of the rows can be used, the next node is the sender of the table. A node therefore can send only the first two columns of its table to any neighbor. In other words, sharing here means sharing only the first two columns.



Step2: Figure 22.16 *sharing distance vector routing to neighbour*



Step3: Figure 22.14 *Distance vector routing tables*



The table for node A shows how we can reach any node from this node. For example, our least cost to reach node E is 6. The route passes through C.

When to Share

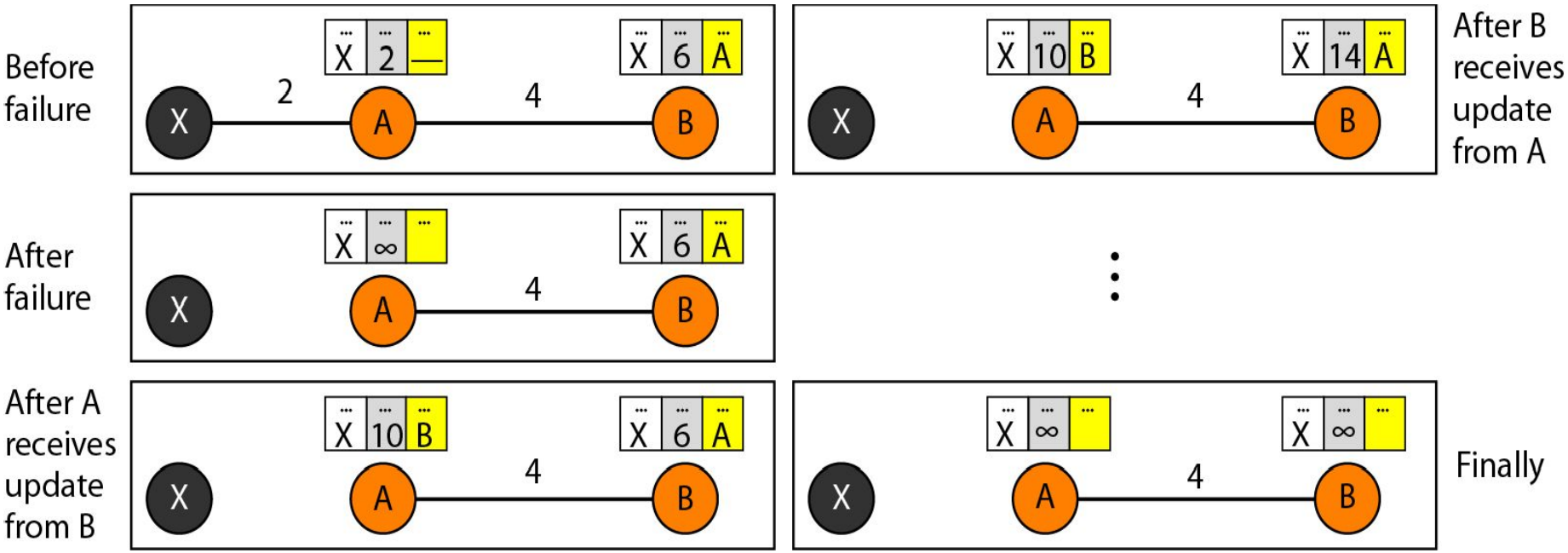
The question now is, When does a node send its partial routing table (only two columns) to all its immediate neighbors? The table is sent both periodically and when there is a change in the table.

Periodic Update A node sends its routing table, normally every 30 s, in a periodic update. The period depends on the protocol that is using distance vector routing.

Triggered Update A node sends its two-column routing table to its neighbors anytime there is a change in its routing table. This is called a triggered update. The change can result from the following.

1. A node receives a table from a neighbor, resulting in changes in its own table after updating.
2. A node detects some failure in the neighboring links which results in a distance change to infinity.

Figure 22.17 *Two-node instability*



A problem with distance vector routing is instability, which means that a network using this protocol can become unstable. To understand the problem, let us look at the scenario depicted in Figure 22.17.

Figure 22.17 *Two-node*

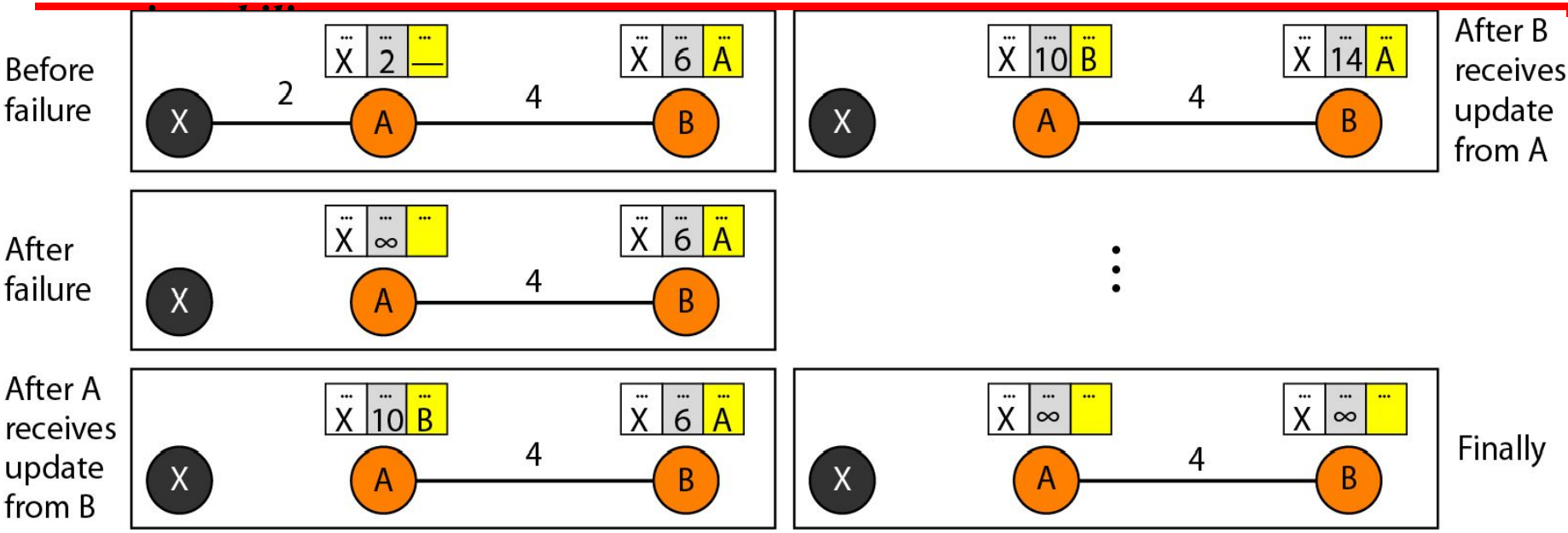


Figure 22.17 shows a system with three nodes. We have shown only the portions of the routing table needed for our discussion. At the beginning, both nodes A and B know how to reach node X. But suddenly, the link between A and X fails. Node A changes its table. If A can send its table to B immediately, everything is fine. However, the system becomes unstable if B sends its routing table to A before receiving A's routing table. Node A receives the update and, assuming that B has found a way to reach X, immediately updates its routing table. Based on the triggered update strategy, A sends its new update to B. Now B thinks that something has been changed around A and updates its routing table. The cost of reaching X increases gradually until it reaches infinity. At this moment, both A and B know that X cannot be reached. However, during this time the system is not stable. Node A thinks that the route to X is via B; node B thinks that the route to X is via A. If A receives a packet destined for X, it goes to B and then comes back to A. Similarly, if B receives a packet destined for X, it goes to A and comes back to B. Packets bounce between A and B, creating a two-node loop problem

Solution to count to infinite problem(loop instability)

1. Defining Infinity :

The first obvious solution is to redefine infinity to a smaller number, such as 100..

As a matter of fact, most implementations of the distance vector protocol define the distance between each node to be I and define 16 as infinity. However, this means that the distance vector routing cannot be used in large systems. The size of the network, in each direction, can not exceed 15 hops.

Split Horizon:

Another solution is called split horizon. In this strategy, instead of flooding the table through each interface, each node sends only part of its table through each interface. If, according to its table, node B thinks that the optimum route to reach X is via A, it does not need to advertise this piece of information to A; the information has come from A (A already knows). Taking information from node A, modifying it, and sending it back to node A creates the confusion. In our scenario, node B eliminates the last line of its routing table before it sends it to A. In this case, node A keeps the value of infinity as the distance to X. Later when node A sends its routing table to B, node B also corrects its routing table. The system becomes stable after the first update: both node A and B know that X is not reachable.

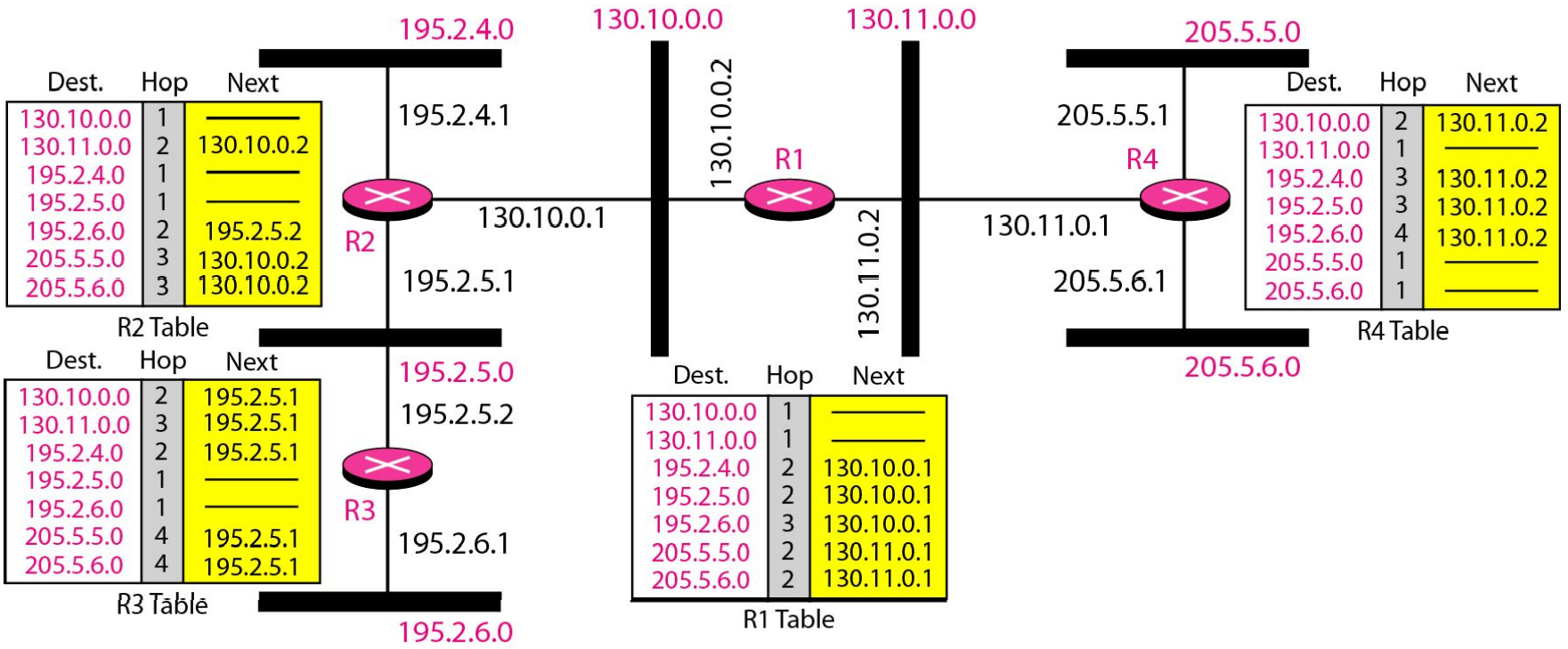
Solution to count to infinite problem(loop instability)

3. Split Horizon and Poison Reverse :

Using the split horizon strategy has one drawback. Normally, the distance vector protocol uses a timer, and if there is no news about a route, the node deletes the route from its table. When node B in the previous scenario eliminates the route to X from its advertisement to A, node A cannot guess that this is due to the split horizon strategy (the source of information was A) or because B has not received any news about X recently. The split horizon strategy can be combined with the poison reverse strategy. Node B can still advertise the value for X, but if the source of information is A, it can replace the distance with infinity as a warning: "Do not use this value; what I know about this route comes from you."

Split Horizon	Poison Reverse
It simply states that you shouldn't advertise a path out of an interface where you first learned the way.	It states that it will advertise that route to you, but with an infinite metric if you've told me this route is not reachable through you.
It is used alone in loop prevention.	It is generally used with Split Horizon for loop prevention.

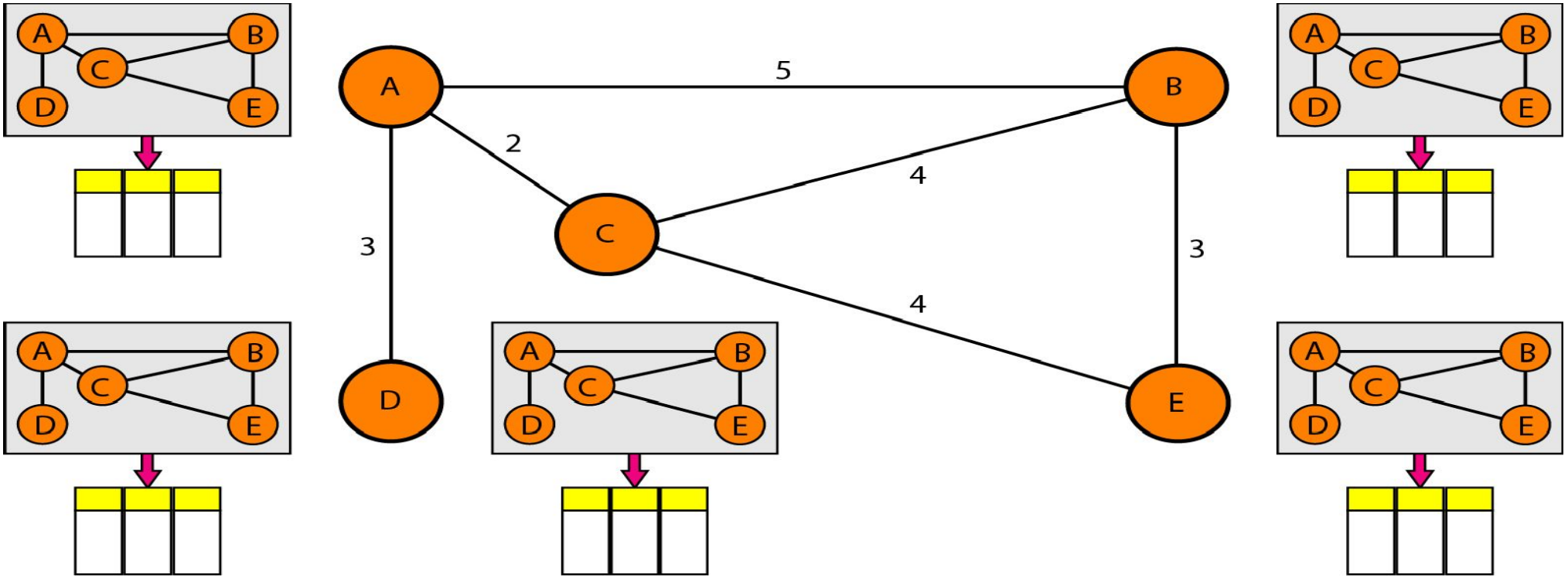
Figure 22.19 *Example of a domain using RIP*



Concept of link state routing

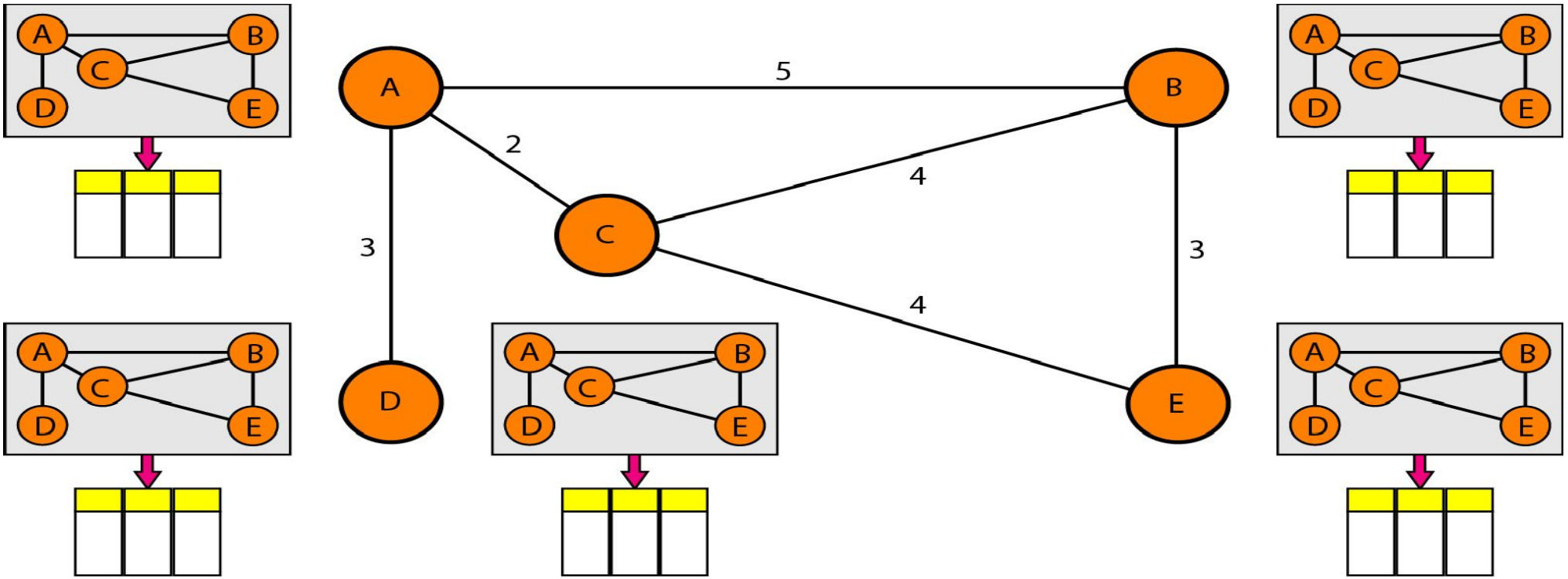
Link state routing has a different philosophy from that of distance vector routing. In link state routing, each node in the domain has the entire topology of the domain the list of nodes and links, how they are connected including the type, cost (metric), and condition of the links (up or down)-the node can use Dijkstra's algorithm to build a routing table.

Link state routing is a technique in which each router shares the knowledge of its neighborhood with every other router in the internetwork.



Concept of link state routing

The figure shows a simple domain with five nodes. Each node uses the same topology to create a routing table, but the routing table for each node is unique because the calculations are based on different interpretations of the topology. This is analogous to a city map. While each person may have the same map, each needs to take a different route to reach her specific destination.



Building Routing Tables

In link state routing, four sets of actions are required to ensure that each node has the routing table showing the least-cost node to every other node.

- **Step 1: Creation of the states of the links by each node, called the link state packet (LSP).**
- **Step2. Dissemination of LSPs to every other router, called flooding, in an efficient and reliable way.**
- **Step 3. Formation of a shortest path tree for each node. Using Dijkstra Algorithm.**
- **Step4. Calculation of a routing table based on the shortest path tree.**

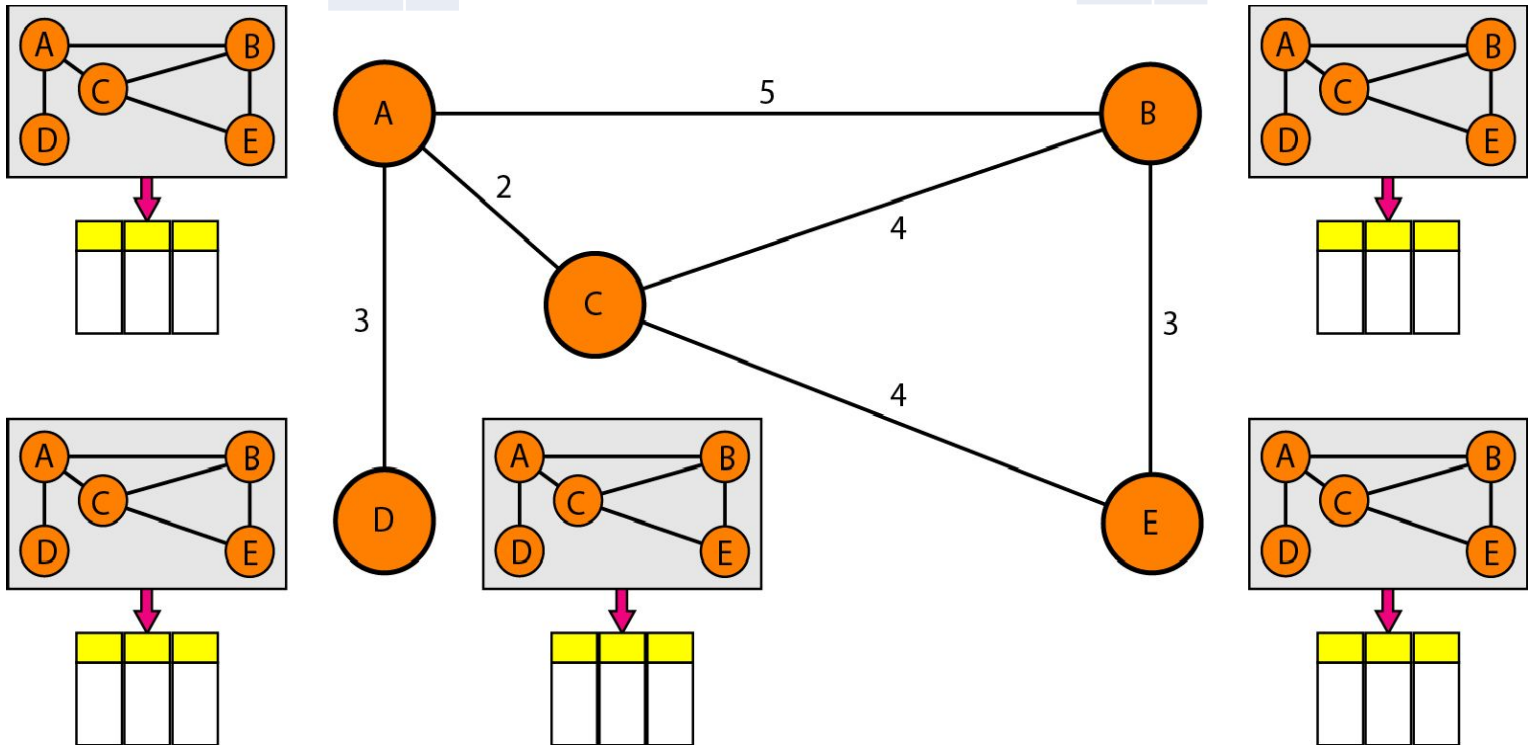
Building Routing Tables

How In link state routing, four sets of actions are required to ensure that each node has the routing table showing the least-cost node to every other node.

Step 1: Creation of the states of the links by each node, called the link state packet (LSP).

A	
Seq	
TTL	
b	5
c	2
d	3

B	
Seq	
TTL	
a	5
c	4
e	3



Step 2: flooding

Flooding of LSPs is done After a node has prepared an LSP, it must be sent to all other nodes, not only to its neighbors. The process is called flooding and based on the following:

- The creating node sends a copy of the LSP out of each interface.

- A node that receives an LSP compares it with the copy it may already have. If the newly arrived LSP is older than the one it has (found by checking the sequence number), it discards the LSP. If it is newer, the node does the following:

  a. It discards the old LSP and keeps the new one.

  b. It sends a copy of it out of each interface except the one from which the packet arrived. This guarantees that flooding stops somewhere in the domain (where a node has only one interface).

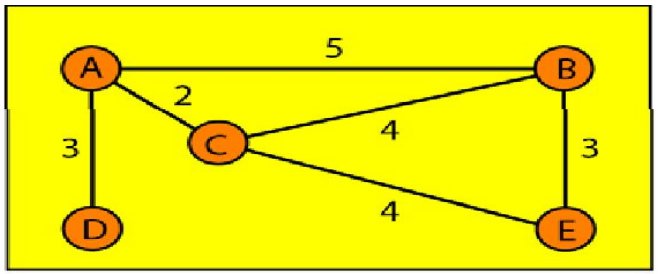
Step 3. Formation of a shortest path tree for each node. Using Dijkstra Algorithm

Dijkstra Algorithm After receiving all LSPs, each node will have a copy of the whole topology. However, the topology is not sufficient to find the shortest path to every other node; a shortest path tree is needed.

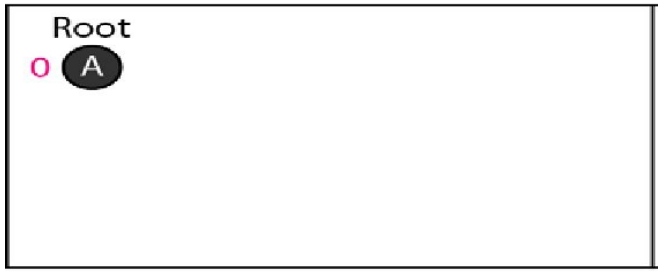
A tree is a graph of nodes and links; one node is called the root. All other nodes can be reached from the root through only one single route.
A shortest path tree is a tree in which the path between the root and every other node is the shortest. What we need for each node is a shortest path tree with that node as the root.
The Dijkstra algorithm creates a shortest path tree from a graph.
The algorithm divides the nodes into two sets: tentative and permanent.
It finds the neighbors of a current node, makes them tentative, examines them, and if they pass the criteria, makes them permanent

Example of formation of shortestpath tree

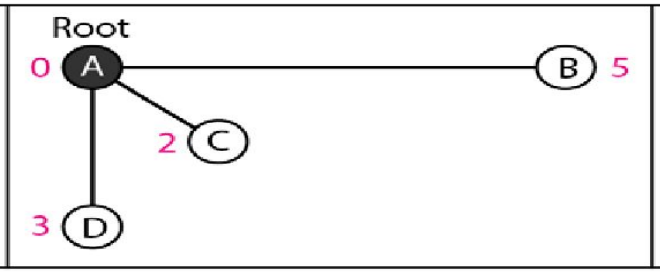
Step 3. Formation of a shortest path tree for each node. Using Dijkstra Algorithm



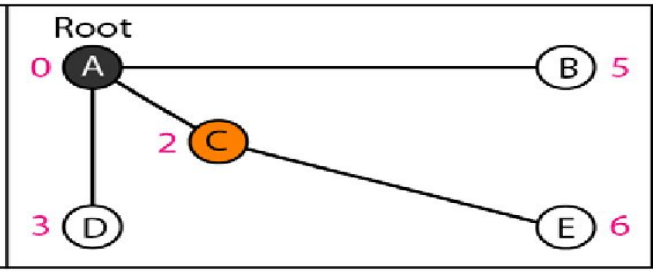
Topology



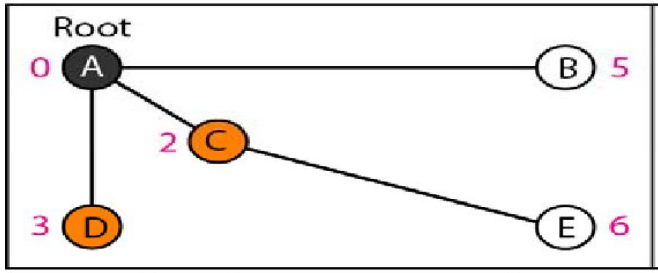
1. Set root to A and move A to tentative list.



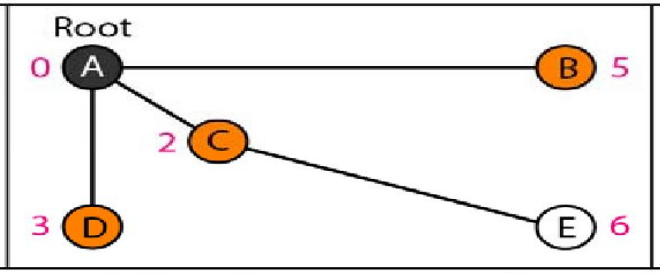
2. Move A to permanent list and add B, C, and D to tentative list.



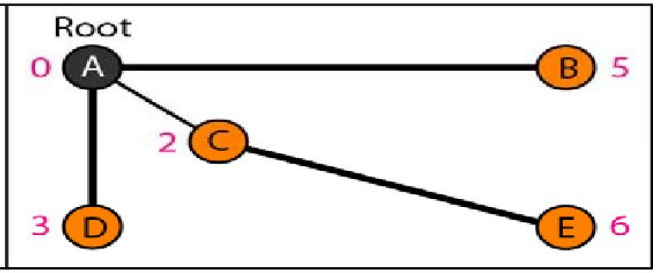
3. Move C to permanent and add E to tentative list.



4. Move D to permanent list.



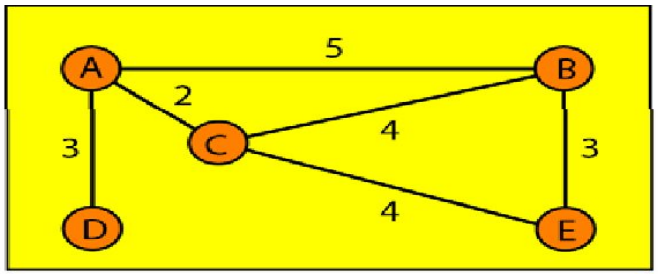
5. Move B to permanent list.



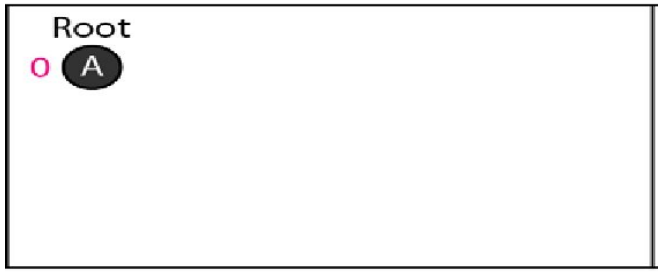
6. Move E to permanent list (tentative list is empty).

Example of formation of shortestpath tree

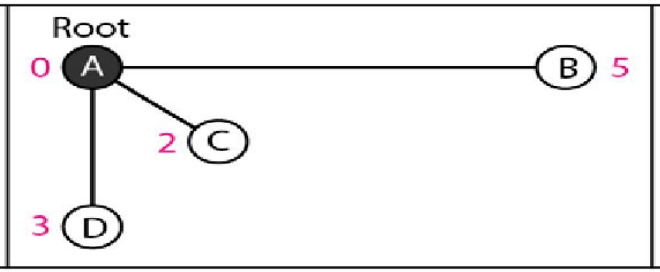
Step 3. Formation of a shortest path tree for each node. Using Dijkstra Algorithm



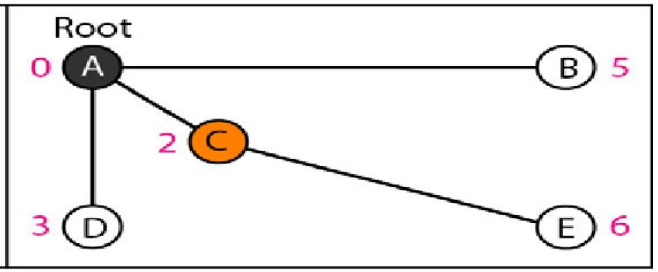
Topology



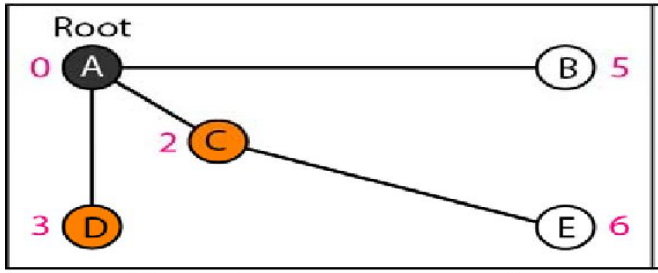
1. Set root to A and move A to tentative list.



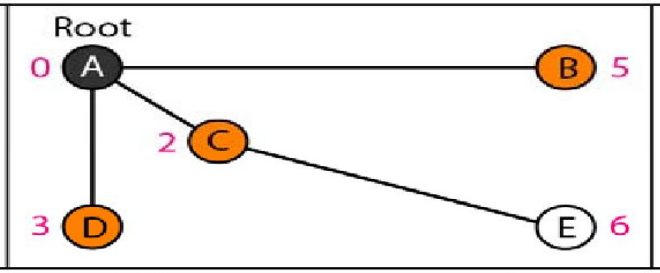
2. Move A to permanent list and add B, C, and D to tentative list.



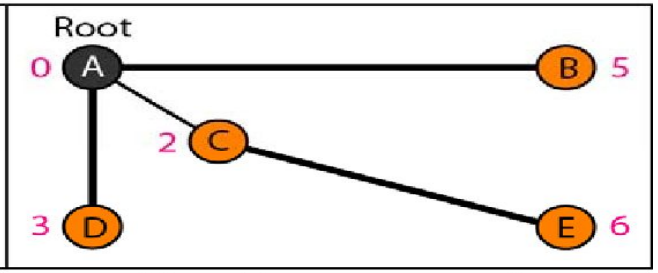
3. Move C to permanent and add E to tentative list.



4. Move D to permanent list.



5. Move B to permanent list.



6. Move E to permanent list (tentative list is empty).

Example of formation of shortestpath tree

Step4. Calculation of a routing table based on the shortest path tree.

.

Routing table for node A

Node	*Cost*	*Next Router*
A	0	—
B	5	—
C	2	—
D	3	—
E	6	C

Flow chart for *Dijkstra algorithm*

